Perceiving and Playing Chess

A Pipeline between Agents and Computer Vision

by

Max Diven

In partial fulfilment of the
requirements for the degree of
Master of Engineering
in
CS408: Individual Project

University of Strathclyde Glasgow

Department of
Computer and Information Sciences

March, 2025

Abstract

This project details the development and integration of a physical-to-digital chess interface, designed to synchronise a tangible board state and the decision making processes of an AI agent. The primary objective was to construct a low-latency perceiving and playing loop capable of real-time board reconstruction and strategic move selection.

The methodology employs a custom computer vision pipeline for square-occupancy detection and a custom AI agent framework for game evaluation. To maintain a consistent architecture a chess engine was built as a meeting point for perceiving and playing participants.

The main findings conclude that while this system achieves high-fidelity synchronisation, it's accuracy and performance are susceptible environmental lighting variance and elusive drifts in sparse end-game configurations respectively. However, the prototype does successfully validate the system architecture, maintaining a seamless digital twin with the physical board state. This provides a scalable framework for future research in Cyber-Physical-Systems and data-centric architectural unification.

Acknowledgements

I would like to thank my supervisor Lee Clift for their continued support, feedback, and guidance during the whole development of the project.

I would like to thank my second marker Jan De Muijnck-Hughes for their insights provided during the initial project progress report.

To my flatmates Callan and Logan. I would like to thank you for your saint-like patience throughout the figurative and literal crashes of the project as well as for listening to my technical ramblings.

Contents

1 Introduction **1**
1.1 Project Aim . . . 2
1.2 Project Objectives . . . 2
1.3 Project Outline . . . 3

2 Background Literature **4**
2.1 Chess as a problem . . . 4
2.2 Chess engine representations . . . 4
2.3 Computer Vision . . . 5
2.4 AI agents and Search algorithms . . . 7
2.5 Overall Pipeline Integration . . . 9

3 Specification & Design **10**
3.1 Methodology . . . 10
3.2 Analysis . . . 10
3.3 Requirements . . . 10
3.4 Design . . . 13

4 Product **18**
4.1 Programming Languages and Technologies . . . 18
4.2 Implementation . . . 19
4.3 Verification & Validation . . . 22

5 Results & Evaluation **25**
5.1 Evaluation . . . 25
5.2 Results of Evaluation . . . 26
5.3 Returning to Objectives . . . 28

6 Discussion & Reflection **29**
6.1 Interpreting the Results . . . 29
6.2 Reflection . . . 29
6.3 Challenges . . . 29
6.4 Limitations . . . 29
6.5 Future Work . . . 30

7 Conclusion **31**

A Appendix **33**
A.1 Full UML Diagram . . . 33
A.2 LiChess Kaggle dataset . . . 33
A.3 Chess.com PGN analyst . . . 33

1 Introduction

The field of artificial intelligence(AI) has largely been concerned with sterile, symbolic environments of logic and pure computation. However, as AI begins to bridge with the physical world, the primary focus shifts from capturing an internal logic to reliable state synchronisation between reasoning some AI agent and information captured by one or more actuators. This dissertations investigates this communication problem of 'closing the loop' specifically between, a continuous physical Computer Vision(CV) model and, a deterministic decision making agent capable of playing chess. When we move from treating a chess board as a static state or node to a dynamic Digital Twin, the need arises for a bidirectional pipeline capable of preserving state integrity while quickly communicating to reasoning processes, despite the inherent latency and unpredictability of capturing a physical environment. In solving this problem we aim to move beyond simple game-playing to persistent real-time interactions between autonomous agents and their analog counterparts.

The significance of sustaining a synchronised state between a physical chess board and a digital agent extends past recreational table-top game-playing. This research mirrors a fundamental hurdle found in fields such as Cyber-Physical Systems as well as Industrial Automation known as the "reality gap", where a perfect synchronisation of state is mandatory, as any dissonance between logical and physical actions can compound into catastrophes. The "reality gap" highlights the utility of chess as a canonical test bed within this engineering context, Owing to the deceptive trajectory of the game. Chess starts from simple initial conditions before exploding into exponentially complex possibilities and representations. By facilitating rigorous state preservation in this controlled, yet high-entropy environment, we develop robust communication protocols and infrastructure required for higher-dimensional state-space complexity.

To navigate this problem, we implemented a decoupled state architecture between the physical board and AI agent. Utilizing modular decomposition we partition our synchronisation problem three distinct sub-problems.

- Perception: Extracting physical data from the board (computer vision system)
- Logic: Enforce the rules and common language of the overall project (chess engine)
- Playing: Analyse and act upon given information (autonomous agent)

This layered approach streamlines design and implementation by establishing clear functional

boundaries that make it easier to tackle our project goals.

1.1 Project Aim

The main aim of this project is to engineer a robust Cyber-Physical framework that sustains the synchronised state of a chess game between a player using a physical board and a neural-inference agent by effectively closing the "reality gap" between the idealistic model view of the agent and the noise that surrounds the physical board.

1.2 Project Objectives

The project is decomposed into the following measurable objectives relative to the layered architecture of the system.

Perception

- Detect the boundary of the physical board relative to the whole image.
- Detect moves made by the player of the physical board
- Discover and Isolate important features of the physical board
- Update internal CV model state according to agent move

Logic

- Provide and communicate a game state in a way is both computational and space efficient to account for the branching factor of chess
- Aforementioned game state representation should facilitate a common language between AI agent and CV model

Playing

- Leverage powerful tree searches and evaluation techniques to ensure moves are computed within a timely manner
- Standardised form of vectorisation to translate board states for agent's neural networks

Overall

- Ensure low latency data transfer that is synchronised across the whole system

1.3 Project Outline

This dissertation is presented as the following sections:

- Section 1: Introduce the foundational context and project motivations, as well as explicitly defined aims and objectives.
- Section 2: A review of literature relevant to three core pillars of the system architecture by evaluating similar existing solutions.
- Section 3: Details the granular design process for the Logic, Playing, and Perception modules. Each requirement is broken down into functional requirements and blueprints. Beyond individual components, this section observes the bigger picture of the overall system integration.
- Section 4: System implementation, design realisations, and technologies, followed by general evaluations with respect to the design
- Section 5: Focusses on prototype and integration testing applied to the system as well as modelling benchmarks
- Section 6: Discussion on the finished product, acknowledges the current limitations of the system as well as possible future implementations to provide a roadmap for future research and development
- Section 7: Closing statement and final remarks

2 Background Literature

Below is a summary of research and existing solutions relevant to this project. Work on decision making systems has always developed along parallel domains, whether that be making the best possible chess move or the identification of pieces on a real board. This project aims to bridge an intersection between these two forms of research to construct a system capable of reasoning about game states from a real environment.

2.1 Chess as a problem

Since the early days of computing research, chess has served as a common benchmark in the field, particularly within artificial intelligence [13]. This is owed due to: the discrete, state-based nature of the game; a strict yet simple set of rules, defining the actions(moves) of any given state; as well as a clear objective in the form of checkmate; and the inherent EXPTIME-completeness of the state-space, starting from 20 legal moves and consistently approaching an order of 30 legal moves until endgames are reached and material falters [11]. Understanding chess from this perspective frames the underlying complexity motivations behind our system.

2.2 Chess engine representations

Chess representations offer a way of encapsulating the board state to a computer and serve as the foundation for digitally modelling the flow of the game.

2.2.1 Bitboards

When modelling the internal state of a chess engine, it is important to use a lightweight and memory efficient representation of the board due to the exponential branching factor of the game. Our engine's internal state will use a data structure called a *bitboard*, a collection 64 bit (unsigned)integers that encode a boolean representation of the game state [8, 12]. Bitboards enable efficient legal move generation through simple bitwise operations, while staying memory efficient when compared to more intuitive matrix representations [8, 12].

For example if we have one of our bitboards represent the white pawns in their starting position it would produce the bitboard: `0x000000F0`

2.2.2 Forsyth-Edwards Notation (FEN)

Forsyth-Edwards Notation(FEN) is a serialisation of the chess board as a string. FEN is designed not as a form of internal state representation but a more exchangeable description of the board [3]. FEN strings are essential in translating board states between the Perception and Action modules [3].

2.2.3 Python-chess implementation

The *python-chess* library is used to represent the systems internal board state as a bitboard, offering FEN string generation as well as bitboard construction from FEN strings [6], simplifying integration of a communication point between the physical system and internal model.

2.3 Computer Vision

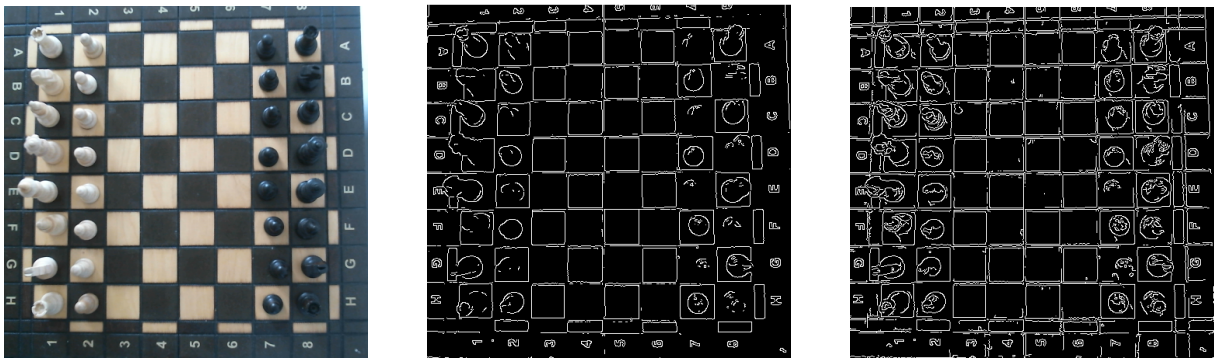
This project will use computer vision techniques and methodologies to interpret the resting and mutated state of the physical board by evaluating its features like piece type or positioning. From this information we can construct FEN strings to be delivered to our chess engine.

2.3.1 Canny Edge detection

Canny Edge detection is a classical computer vision algorithm that extracts/highlights the edges of objects within a frame, to a manually tuned degree of activation [4]. The algorithm remains a top option during object image detection as a result of it's Gaussian pre-processing pipeline, which removes substantial noise from the frame, making it easier to process [4, 10].

2.3.2 Auto Canny

Auto Canny is a similar pre-processing algorithm that typically yields a higher degree of fidelity [10]. While standard Canny Edge is powerful it is not entirely rigorous to environmental factors, such as lighting. Due to the detection threshold being manually tuned [4] regular Canny Edge may suffer from magic number syndrome [10]. Auto canny fixes this issue by dynamically calculating this threshold at runtime based on the median pixel value of the image. We use this because it extracts edges while still mostly preserving object correlation in space [10].



(a) Original Frame (b) Standard Canny (Fixed) (c) Auto Canny (Robust)

Figure 1: Comparison of edge detection methods compared to original frame, a

2.3.3 YOLO and Piece detection

YOLO or, 'You Only Look Once' is a specialised convolutional neural network (CNN) built around grid-based image classification, separating background and foreground features [9]. Upon observation this makes YOLO an intuitive choice for piece detection over a physical board [5]. However, YOLO inherently comes with an add a layer of complexity when it comes to facilitating the data and computational overhead required to effectively train the neural network.

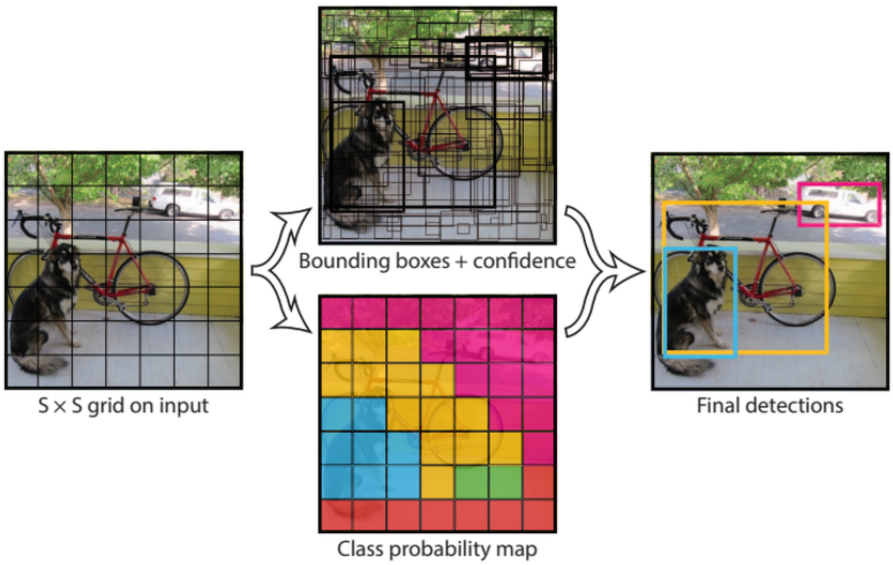


Figure 2: YOLO grid model separating foreground images from the image background [9]

The research presented around YOLO is based YOLOv8 due to its recency and attention based optimisations over previous iterations. Although multiple piece detection prototypes exist on previous

versions, YOLOv8 is one of the most consistent and accurate model to date [5]. As shown in (Fig. 3).

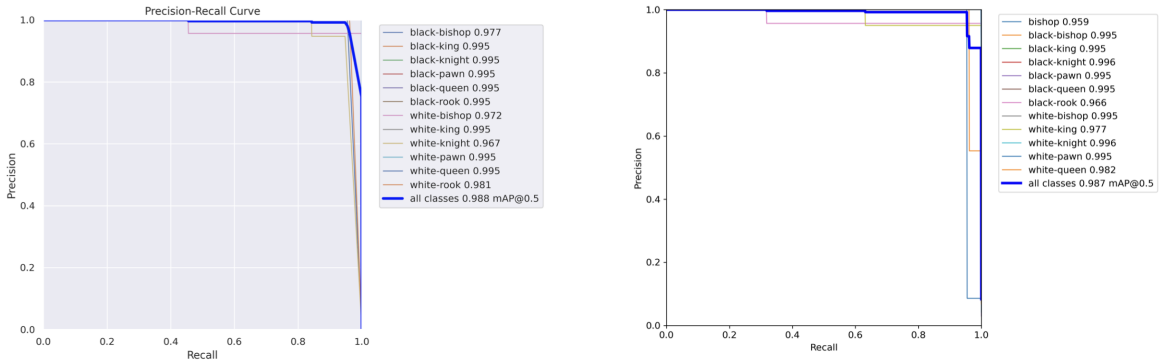


Figure 3: Comparison of YOLOv8(left) vs YOLOv7(right) at Precision Recall [5]

2.4 AI agents and Search algorithms

When developing AI agents capable of playing chess it is important to start from strategies built around game-playing tree searches. The two most common being Minimax [8, 11, 13] and Monte Carlo Tree Search (MCTS) [7, 14]. While both search methods aim to select the most optimal move according to some evaluation [7, 8], it remains important to distinguish these search methods by the strategies they apply as they tend differ quite drastically.

2.4.1 Minimax

Minimax is a recursive search algorithm that exhaustively explores future game states to select the best possible action, maximising win potential for the player and minimising it for the opponent over each turn of the game [13, 11]. The reductive process of move selection can be extremely slow, especially considering the branching factor of chess [11]. The base minimax implementation serves as great foundation for decision making processes, yet in pragmatic application minimax is extremely dependent on heuristics, Alpha-Beta pruning, and iterative deepening to efficiently guide the search [13].

Alpha-Beta Pruning

Alpha-Beta pruning is a simple optimisation of the minimax search algorithm that prevents the exploration of redundant branches [8]. Redundant exploration is prevented by two bounds: α, the lower bound of the maximising player; and β, the upper bound for the minimising player. When the

search assesses that a node value cannot fall within the α-β window, it's branch is pruned and can no longer influence the final outcome of the search [8].

Pruning reduces the complexity of the search from $O(W^d)$ to $O(W^{d/2})$, where W is the branching factor, and d is the maximum depth of the root respectively. This effectively enables minimax to search twice as deep, given the best case scenario [8]. This optimal search complexity can more consistently be achieved by applying iterative deeping to effectively order moves [13].

Iterative deepening

Iterative deepening explores next states at incremental depths, using evaluations from shallower depths to identify and order promising moves, with their values memoized to optimise computation time over further searches [8].

2.4.2 Monte Carlo Tree Search (MCTS)

While minimax relies on exhaustive exploration, MCTS is a stochastic search that relies Markov Decision Processes (MDP) to construct and traverse a tree, adhering to some evaluation function [7, 14]. In our case our tree traversal is based off game analysis from a small pre trained neural network [14]. Most MCTS plus neural network applications follow the AlphaGO approach, guiding the search using a policy and value headed neural networks [14]. both networks return probabilistic outputs relative to the stage of exploration. The policy network returns a probability distribution of unexplored states' chances of being a good choice, pruning less promising child nodes. On the other hand, the value network returns a scalar probability indicating the likely hood of a child winning [14]. However powerful these 'assistant' networks are, they do pose added complexity and challenges.

Deciding the best move from any given state is based off some Upper Confidence bound for Trees(UCT) function that balances the current exploration of the tree against potential moves assessed by the neural network to build trees much quicker than deterministic searches [14].

for our evaluation we will use $PUCT$, The formula employed by AlphaZero models [13, 14]:

$$PUCT = Q(s,a) + c_{puct} \cdot P(s,a)\sqrt{\frac{\sum_b N(s,b)}{1 + N(s,a)}}$$

where:

- $Q(s, a)$ = The current exploration value given by the value head of the neural net at a given state
- $P(s, a)$ = Policy value of the current node we are at, usually the probability that the next move is good
- N = the number of simulations from some chosen node of the tree
- c_{puct} = exploration constant, where $c_{puct} \in \mathbb{R}^+$

2.4.3 Parallel Search

Tree searches allow us to consider many possible games. However, they all explore the tree serially, resulting in extremely slow performance over large state spaces [8]. There are many forms of parallelisation that can be adopted to speed up tree searches specifically, some include: leaf, root, and tree based parallelisation respectively [14]. This use case is best suited root based Lazy Symmetric-Multiprocessing (LazySMP) [12] approach for both Minimax and MCTS, which spins up a search process for each child of the root as a runnable for some executor service. This multithreaded approach aided by memoization of relevant hash tables can drastically speed up each tree search [8].

2.5 Overall Pipeline Integration

The pipeline is responsible the transfer of FEN strings between the chess engine and CV model, allowing them to interface with one another using a common language [3]. FEN string generation is trivial for our chess engine [6]. As for our computer vision model, piece identification can be handled by YOLOv8 [5] or classical edge detection methods [4, 10] which may prove useful in construction of an inference model with relative confidence. Challenges begin to arise around piece detection models when considering each square on the board as a sort of feature.

Prior pieces of work have tried to solve this problem by training a CNN to view the board and interpret the individual state of each square, sequentially generating a FEN string after evaluating each square [3]. The CNN is decently accurate but not free from small consistency errors [3], making it less considerable due to the complexity. However, the idea of zoning squares may prove useful [3].

3 Specification & Design

3.1 Methodology

We approached our pipeline problem using an iterative waterfall methodology for the entire lifecycle of this project. Iterative waterfall was chosen on account of the structure and flexibility it provides compared to other methodologies when leveraging the phase-gate linearity associated with the traditional waterfall alongside flexible iterative cycles for technical refinement.

To realise our methodology in execution, we utilised the Kanban framework to maintain order during development. By decoupling the three primary sub-problems into sequential modules, we enforce linear progression of the project's macro-architecture whilst allowing structured iterations in each micro-block of development. Ultimately this modular execution ensured that the technical complexities of any given component in the pipeline did not impede the implementations of any others, facilitating a stable foundation for the final software integration.

3.2 Analysis

The motivation to ground this pipeline around the deterministic, rule bound environment of chess was driven by the unique 'closed-loop' system it provides. Unlike generalized object detection, every output from the CV module must map to one legally available move from the currently observed board state from the chess engine. This deterministic condition directly informed the design specification. By analysing potential input for errors or differences, we determined the engine must act as a translator between raw image data and the AI agents actions. Consequently, the project was partitioned into three autonomous functional modules. This enabled us to treat each module as discrete service, capable of performing jobs within the greater system.

3.3 Requirements

Each requirement of the system is assigned to one fo the three aforementioned functional modules. To ensure clear allocation, some requirements may carry dual-component specifiers (e.g UI/Logic) where dependencies have separate components that fall under the same label. The priority of each requirement is categorised using the **MoSCoW** (Must, Should, Could, Would) framework.

3.3.1 Functional Requirements

Below is the table categorizing the conditionals outlined above

Index	Constraints	Module	Priority
FR-01	User must attach a peripheral camera, placed at a birds-eye view over the board	Perception	Must
FR-02	Must constantly display capture frame of the board	Perception	Must
FR-03	Should allow user to specify the angle of the camera relative to the board	Perception	Should
FR-04	Must allow user to define an 8×8 grid under variable lighting, using the mouse.	Perception	Must
FR-05	Must facilitate input loop for user to communicate moves with the engine.	Perception	Must
FR-06	Board orientation should be drawn over the frame of the physical board	UI / Perception	Must
FR-07	Constantly listen for updated state from partner module without interrupting either main loop	Logic & Perception	Must
FR-08	Must map occupancy data to a valid FEN string via move history.	Logic & Perception	Must
FR-09	Must serve non-human digital inputs through an `Agent` interface.	Logic	Must
FR-10	Should be able to run without the engine component and receive digital input form a human (e.g. mouse input)	Logic	Should
FR-11	Must use lightweight state representation for the chess board that allows the creation of game trees	Logic	Must
FR-12	Must legally enforce game state against the rules of chess	Logic	Should

FR-13	Should allow for moves to be undone	Logic	Should
FR-14	User could manually override square occupancy via a GUI fallback.	UI / Logic & Perception	Could
FR-15	Must sustain a game instance of chess that is reactive to user and agent input	Logic	Must
FR-16	Must be able to construct a game playing tree occupied with game state nodes	Playing	Must
FR-17	Agents should be able to make moves within a reasonable time frame	Playing	Should
FR-18	Must be able to vectorise internal engine board representation into a tensor understood Agents evaluation networks	Playing	Must
FR-19	Must differentiate pieces from noise when analysing images of the board	Perception	Must

3.3.2 Non-Functional Requirements

Below is a similar table to that found within Functional Requirements

Index	**Constraints**	**Category**	**Priority**
NFR-01	Latency improvement: AI should respond fast to keep game uptime maximal (around ≤ 5 seconds)	Performance	Should
NFR-02	Capture analysis could be good enough to operate in uncertain environments (shadows, sharp lighting, etc.)	Robustness	Could
NFR-03	Should run on a consumer grade laptop and not require a gpu	Portability	Should

3.4 Design

3.4.1 Interface Design

The systems user interfaces were written as a utility to bridge the gap between raw data processing, validation, and general interfacing with the logical state. Instead of implementing one monolithic UI, we utilised two distinct environments tailored to the specific needs of the Logic and Playing, as well as Perception modules respectively.

Chess Engine View

To test Logic and Playing Modules independently of the camera hardware, a minimalist Pygame interface was developed as a Digital Twin of the chess board. The system validates and abstracts board information received from the Perception component or an autonomous AI opponent, before rendering a synchronised digital frame. This dashboard provides a clean environment for move input and engine evaluation, offering a significantly more legible interface for debugging than terminal output or raw FEN strings. Crucially this abstraction simplifies integration of various AI agents, given they adhere the standardised control interface defined by the engine.

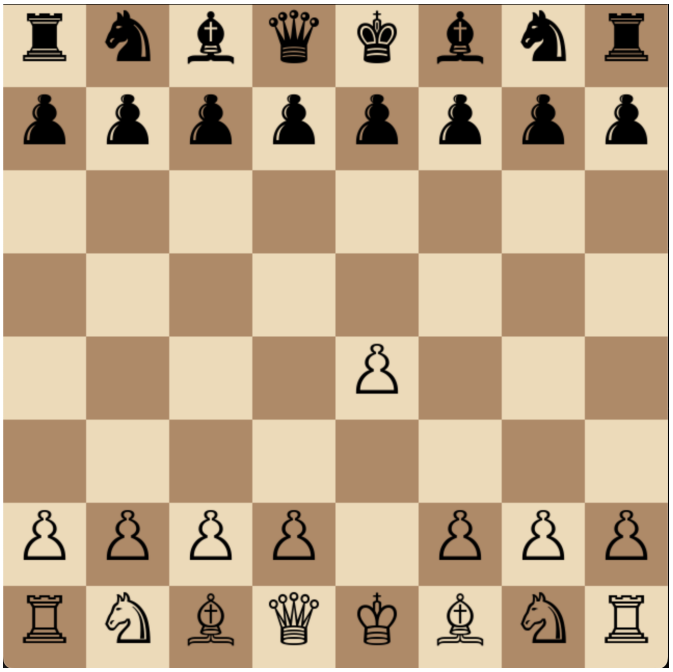


Figure 4: Engine dashboard after the first move of the game

Perception Capture Projection

The Perception interface evolved from static image prototyping to a live-feed, Augmented Reality (AR) overlay. By projecting the internal 8x8 rank and file mappings of the physical board directly onto the camera frame. The system provides immediate feedback on grid alignment and occupancy detection, allowing the user to quickly verify the model's interpretation is grounded before sending an instruction to the chess engine.

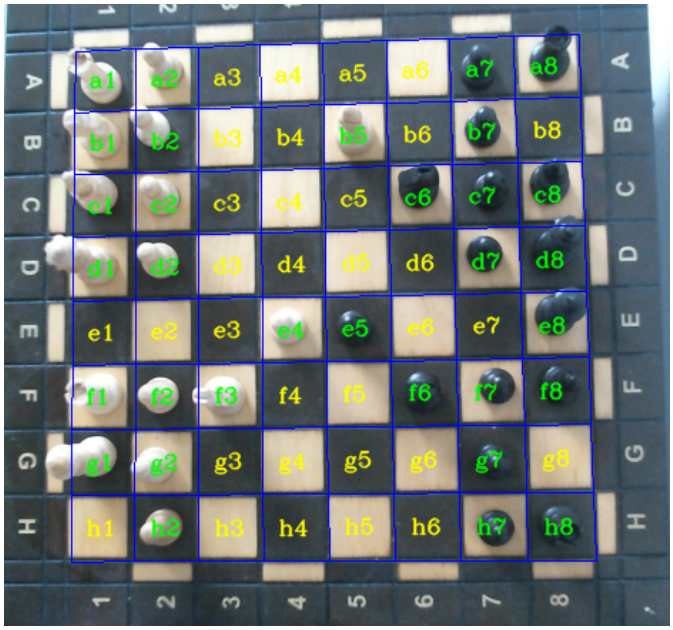


Figure 5: AR overlay displayed over capture frame

3.4.2 System Design

To transition from conceptual requirements into a system design we constructed a Component model, focussing on the communication between the physical sensing and logical processing. The system employs a Broker pattern, where the Logic component mediates a conversation between the Perceiving and Playing modules. This decoupling ensures that raw sensory data is translated into high-level states before formulating playback sequences, allowing for hardware independence and modular scaling during development.

To visualise the boundary between these models, (Fig. 6) provides a structural overview of the system architecture. The internal relationships of the primary components can be seen below.

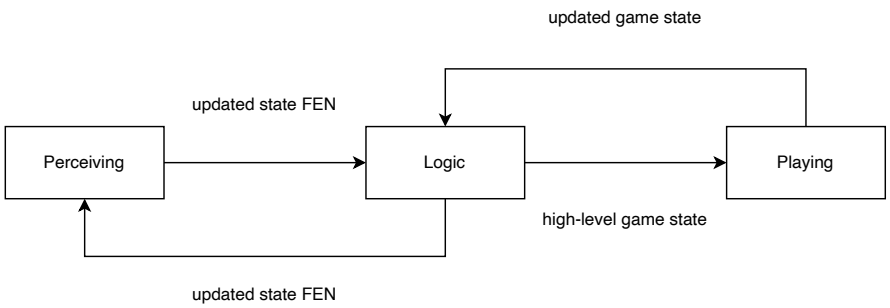


Figure 6: High-level diagram of communication flow within the system

As we can see illustrated above the Logic component acts as an effective conversation point of the system, decoding data from the Perception component and encoding data from the Playing module. By Isolating these processes, the design ensures the core decision making logic remains agnostic to any camera hardware peripherals used.

The following divulges into more concrete designs for each of the illustrated components, detailing internal state logic, data structures, and specific interface protocols required for implementation.

Logic

At a high-level the logical engine acts as a simple Model-View-Controller(MVC) architecture decoupling game logic from UI, enabling efficient state management and real-time visual updates.

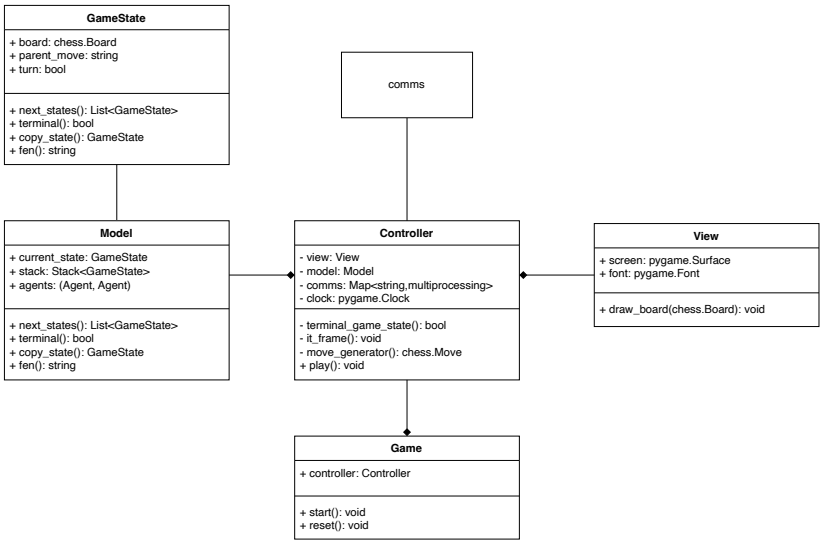


Figure 7: UML diagram of the Logic component of the system

Above we can see that, the `View` is responsible for rendering a more digestible interpretation of the internal state (see Fig. 4). Conversely, the `Model` facilitates the mechanics of chess, providing an interface to act upon and modify the `GameState` object. In coordination, the `Controller` manages communication of these components. Finally the `Game` object encapsulates all of this logic into one configurable entry point that is flexible for testing and interpretation (Fig. 7).

Playing

The Playing module interacts with the Logic component through the `Agent` interface. As long as some `Agent` can provide a '`move`' method, accepting of a `GameState` object and returning a modified one, the engine will recognise the `Agent` object's "ability to play". Below is a UML diagram for a specific implementation, the Neurally Assisted or `NA_Agent`.

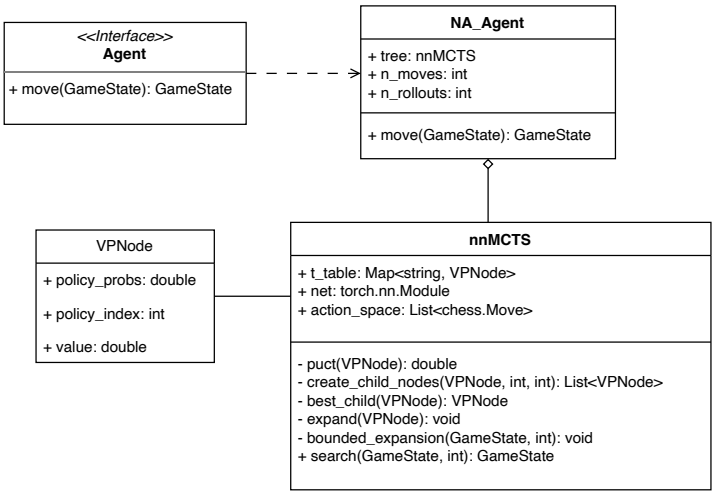


Figure 8: UML representation of `NA_Agent`

While `NA_Agent` utilises a search tree during move selection, a common trait among autonomous agents, this is not a requirement of the underlying interface. The `Agent` abstraction remains agnostic to the internal decision-making process, allowing for diverse implementations.

Perceiving

While the Logic and Playing modules are decoupled through interface, they share the same high frequency execution context within the gameplay loop. In contrast, the Perception module operates as a true 'outsider' to this internal dialogue. By running as a separate system process, the Perception

module is physically separated from the internal logic, much like the physical board. This architectural boundary ensuring core logic remains unburdened by high intensity environment of feed capture and image processing.

To bridge this interprocess gap the system employs a `CameraAgent` to act as a listener for actions performed by the Perception module. This satisfies the `Agent` interface, allowing the `Controller` to wait for a move as if it were a regular engine participant.

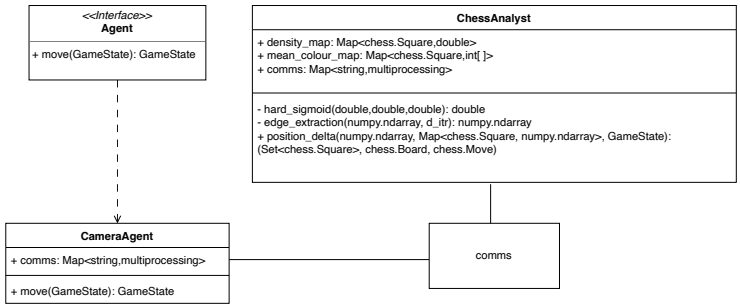


Figure 9: UML interpretation of main Perception conversation

The use of an `Agent` type listener coincides with the modularity of the system; the `Controller` does not need to implement specific logic for handling process communication and synchronisation. Instead, the complexity of the external process is encapsulated within this listener, facilitated by the `comms` object. This 'stalling' mechanism is a deliberate design choice that treats physical observations as a standard asynchronous move provider, ensuring the Logic component's inputs remain uniform regardless of the source.

General Architecture Overview

The high-level specifications within this section detail the system's architectural blueprint. By establishing clear boundaries between the Logic, Playing, and Perceiving partitions, the design ensure a modular environment where the engine remains indifferent to the source of a move.

It should be noted that the structural relationships demonstrated in figures 7,8, and 9 represent a simplified view of the whole system architecture (Fig. A.1). These sub-diagrams exist to illustrate the most important pillars of the system, as these corners are where most of development will be focused.

4 Product

Realising a functioning system from modular designs requires a symphony orchestrated by high-frequency image processing and deterministic game logic. To achieve this, the abstract boundaries defined by the Design (Section 3.4) were bound to a high-concurrency software environment. This section details the technical rationale behind the realisation of the pipeline and its 3 main pillars.

4.1 Programming Languages and Technologies

4.1.1 Programming Languages

This system was programmed entirely using Python 3.14, initially this was to leverage the experimental Global Interpreter Lock(GIL) free build to achieve faster execution speeds. However, the project changed back to the standard CPython build for long-term stability and library compatibility. Despite the interpreted nature of Python, it was chosen over something like C++ for this latency sensitive implementation due to its ease of interfacing as well as extensive ecosystem of high-performance libraries

4.1.2 Libraries

Below is a list of the aforementioned libraries this project employs

- **python-chess**: A simple pure Python library that works in any Python 3.8+ environment; facilitates the system with bitboard state representation (2.2.1), board action space, as well as FEN string generation.
- **pygame-ce**: PyGame Community Edition is the current maintained continuation of PyGame, chosen for its broader compatibility when compared to regular PyGame versions. This library is the foundation of our Logic module's MVC architecture described in Design (3.4.2).
- **numpy**: The standard for linear algebra interfaces in Python. Achieves efficient linear operations by utilising stride architecture with C-implemented backing arrays. Used during this project for the vectorisation of states as well as construction of datasets used to train our Neurally Assisted Agent (Fig. 8)
- **PyTorch**: The 'De Facto' solution for constructing and training neural networks. Testing and re-evaluating can be done quickly on account of the library's GPU compatibility. Applied in

constructing and training the Neurally Assisted Agent.

- **polars**: Data analysis library known for its lazy evaluation when scanning documents, enables filtering of information before fully reading it into memory. Proves particularly useful in speeding up dataset construction.
- **OpenCV**: The industry standard framework for real-time computer vision systems. Built upon fast and reliable C foundation, it serves as the backbone of the Perception module, from frame capture to classical processing algorithms like canny edge.

4.1.3 Technologies

While the systems implementation is defined(in-part) by its libraries, the role of external technology and tools cannot be understated. Below are tools that were utilised in streamlining the workflow over the entire project lifecycle.

- **Git**: Git is the definitive version control software, owed to its 95% market share [1]. Git enables almost frictionless feature implementation during iterations of development, this can mainly be attributed to branches that were used for the development of each feature.
- **GPU acceleration**: This project leveraged GPU hardware acceleration when training our neural networks. The parallel execution of linear algebra operations offered significant training speed enhancements, reducing training time from multiple hours to mere minutes and allowing more consistent testing and iteration phases.

4.2 Implementation

The implementation adheres to the modular structure established in Design 3.4.2 for all stages and iterations of development. This creates a strong foundation to build and refine upon while preserving architectural boundaries.

4.2.1 Logic

Development of the Logic component was the most straight forward and linear according to the design (see Fig. 7). The first thing implemented was the `GameState` object, acting as a wrapper of the python-chess `Board` object abstracting some conditionals and limiting actions from any given state to the purposes of our engine.

After building the `GameState` we could tackle the realisation of our proposed MVC architecture. The `View` was constructed only using the pygame display, along with some geometric intuition surrounding the board. When rendering our internal state, the pieces are represented by the appropriate unicode chess character.

Due to the composite nature of the design, the next object to implement was the `Model`. To facilitate the mechanics of chess the `Model` keeps track of turns and updates the `GameState` depending on human and `Agent` moves.

Finally the `Controller` implementation sustains the main engine-loop and acts as the glue between the `View` and the `Model`. From here, we can construct a `Game` object as a configurable entry point for the MVC architecture.

4.2.2 Playing

After constructing Logic engine foundation, the Playing module was mainly developed around two `Agent` types capable of interacting with chess engine. The Heuristic Assisted or `HA_Agent` (detailed in A.1) and the previously mentioned `NA_Agent` (see Fig. 8).

The first to be implemented of the two was the `HA_Agent`, centred around the `Minimax` game-playing tree with some chess board scoring function. In our case this is a balanced sum of material, mobility, and centipawn evaluations respectively. Due to the branching factor of chess and exhaustive nature of `Minimax` the search had to be optimised for making moves within a reasonable time frame. This was achieved using the GIL-free build of Python 3.14 in tandem with iterative deepening and the Null Move Heuristic(NMH) to quickly compute and prune moves of the tree. By spinning up an executor service we off load depth limited searches for each child of a given state, this results in a significant speed up when all threads share a single transposition table allowing them to effectively mark inefficient paths for each other. Stacking iterative deepening as well as the NMH on top of this shared memory model creates more effective memoisation, significantly reducing our compute time and allowing us to scale for mid-game board states that possibly branch into an average of 40 children.

Developing the `NA_Agent` required us to build an external chess prediction model that could be applied in conjunction with a `nnMCTS` tree search to, with the intent of evaluating the best legal move to take from any given state. The prediction network consists of a convolutional trunk to

extract spacial information form the board, a feed forward value head to asses whether the agent is on a winning path, and a feed forward policy head to output a probability distribution over all legal moves. To build a dataset that could be recognised by this neural network, we started by vectorising an existing dataset of 20,000 chess games (found at. A.2). This involved processing all board states described by the move history for any given entry within the original dataset into a $13 \times 8 \times 8$ tensor encoding as well as creating value and policy labels. Each outer slice represents the positional information of the 8×8 board for all 12 piece types(6 black, 6 white) as well as one slice for rules like castling or en passant. With the vectorised dataset model training could begin using a train, test, validate split. Once the prediction model was deemed satisfiable, the neural weights could be extracted and quickly reconstructed in the full system by the `NA_Agent` at runtime.

To use the chess prediction model, we pass our internal state through the same vectorisation function used during dataset construction and parse that as input to our model. The `nnMCTS` then builds a markov chain to select the most optimal game path from a given state by weighing the stochastic output of the value and policy heads accordingly against the $PUCT$ formula (see Background Literature 2.4.2), resulting in rapid and effective move selection.

4.2.3 Perception

The engineering and assembly of the Perception module began with simple image prototyping, allowing the user to define the corners of the board before constructing and rendering an internal 8×8 grid of the squares relative to the rotation of the board from the perspective of the camera. This initial derivation of spatial logic served as the benchmark for the instantiation of live-feed experimentation. Once consistent configuration and performance were established the next phase of Perception development involved creating a system capable of recognising human-made moves over physical board. To achieve this, an inference model was built within the `ChessAnalyst` (Fig. A.1) to predict the most likely candidate move based on users actions over the board relative to the system's last known logical board state. The `ChessAnalyst` is centred on predicting user input structured on image comparison between frames focussing on each square of the internal grid. Normal moves were evaluated using differences in Auto Canny density for any given square S to prevent noise from creating false positives. The evaluation employed the following product, acting as a pseudo **AND** gate to zero out noise caused by board artifacts or lighting:

$$\Delta = |S'_{to} - S_{to}| \cdot |S'_{from} - S_{from}|$$

Original position difference weighting used to identify normal moves

This square evaluation can accurately estimate piece transpositions caused during normal moves. For capture moves, a heuristic was developed around the average colour value of the pieces. Capture moves can only occur between opposing pieces meaning colours will always contrast, creating largest discrepancy of all capture move candidates. This can be expressed simply if S is the colour value of a given square, such that:

$$\nabla = |S_{from} - S_{to}|$$

Absolute difference of colour value per given square, used to identify captures

With this established criterion, the Perception module can switch between evaluating captures and normal moves based on membership of a normal move set, flagging capture assessment if the aforementioned set is empty.

4.2.4 Pipeline Implementation

Integration of the overall pipeline was mostly achieved using Python's multiprocessing module. It started with the instantiation of the `comms` object, a map containing two blocking queues to facilitate a conversation between the Logic and Perception modules as well as a halt flag to fully collapse the two processes given one is shutdown. Each queue is responsible for transferring the FEN string and parent move of the estimation given by the engine or computer vision system. This enables a fast and consistent transfer stream over a common language that allows us to identify construct appropriate internal states. If the user chooses to play using a camera, `comms` is constructed and passed down to each process, with a `CameraAgent` being instantiated as a listener within the engine.

4.3 Verification & Validation

The verification and validation of the system were conducted through a prototypical integration cycle, prioritising end-to-end integration over isolated unit testing. This approach was necessitated by the concurrent and non-deterministic nature of the hardware-software interface, where the systems

success depends on the real-time synchronisation of the Perception and Logic modules.

4.3.1 Verification: Prototypical Debugging

Verification was achieved through incremental functional testing within a 'closed-loop' system. Rather than utilising abstract unit tests, each module was verified by it's ability to remain state consistent under live conditions.

- **Perception Verification**: Verified testing the internal 8×8 grid against the physical board state, progressing over 30 random moves(15 half moves) to ensure the Digital Twin correctly inferred piece position.
- **Logic Verification**: Verified by feeding the engine FEN strings and observing the correlation between state construction and the rendered output, as well as testing user input against AI agents
- **Playing Verification**: Verified by playing against agents in using the engine UI and comparing their move selection with classical optimal moves(openings, trades, etc.)
- **Neural Network Verification**: Assessed directly using train, test, validate split when training models, measuring value head, policy head, and value sign accuracy respectively.
- **Concurrent Verification**: Debugged by monitoring the 'Handshake' between the Perception and Logic processes, ensuring no deadlocks, race-conditions or synchronisation issues occur during queue polling.

4.3.2 Validation: Requirement Traceability

Validation ensures the final build satisfies the functional and non-functional requirements of the system defined in (Requirements 3.3). Below is a table detailing the functional and non-functional requirements that were achieved.

Index	**Validation**
FR-01 - FR-06 & FR-14	Validated implicitly through operantionalisation of the live capture feed. The achievement of a stable capture loop that was agreeable to user input served as a composite benchmark without requiring isolated test-beds.

FR-07 - FR-08	Validated against extensive runtime and action of the overall pipeline integration without either process interrupting each other.
FR-09 - FR10	Both requirements are validated with the existence of the `Agent` interface. The engine behaves the same independent of the camera peripherals
FR-11 - FR13	Validated implicitly through `GameState` object as well as backing python-chess board representation.
FR-15	Validated during initial development of the Logic component and still holds even after construction of the Perception module.
FR-16	Validated during prototypical testing of `NA_Agent` and `HA_Agent` respectively.
FR-17 & NFR-01	Validated during testing of the overall product when `NA_Agent` is present, with an average response of less than a second.
FR-18	Validated during implementation of the chess prediction model as was necessary.
FR-19	Validated through tuning of the normal move detection system using the defined delta product (see Implementation 4.2.3).
NFR-03	Validated by the construction of prediction network using pre-saved weights.

4.3.3 Analysis of Deviations

While most requirements were met naturally throughout development, the system encountered marginal constraints regarding NFR-02 and lighting robustness. The system struggles when presented with imbalanced lighting, which creates false positives when evaluating canny edge density due to shadows introducing enough noise to pass the detection threshold.

Furthermore, FR-13 was only partially achieved due to process context overlap. The concurrent relationship between the Logic and Perception modules produced challenges on deciding an appropriate input context for an undo action. The reduced implementation is exclusive to the Perception module, allowing the user to undo mistakes made by the inference model before alarming the engine of the updated state.

5 Results & Evaluation

To transition from a working system to consistent system necessitates a comprehensive testing framework to validate the architectural integrity of the project. This section outlines the testing used to evaluate agent response time, by monitoring real-time decision making. Neural network testing accuracy to ensure robust logical evaluation of external information. This also includes tests of the whole system control flow, analysing the quality of agent responses and computer vision integrity simultaneously ensuring system robustness.

5.1 Evaluation

This section outlines the methods employed by the system to fully evaluate the criteria established above.

5.1.1 Agent Speed

The goal of measuring the agent response speed of the Playing module arises from a demand to ensure real-time execution with accuracy, as not to ruin the flow of the game or render the system redundant in the context of application.

Agent speed will be tested by making our two `Agent` implementations play against each other for two games, allowing each agent to play white once. Each game will consist of 40 moves to guarantee intense mid-game states, whilst also ensuring there are enough samples to measure the quantitative results of each agent.

5.1.2 Neural Network Accuracy

To evaluate our neural network accuracy a train, test, validate split will be applied over an iterative range of 5 - 10 epochs of training. This phase of evaluation monitors computational overhead during training as well as the overfitting threshold of the prediction model. This method of evaluation aims to move beyond simple functional parity to asses operational excellence and applicability in real-time, 'high-stakes' environments.

5.1.3 Overall system evaluation

The overall system was assessed using a scenario based end-to-end test modelled after Morphy's Opera Game (1858), a well known chess game demonstrating rapid piece development and a forced checkmate [2]. This benchmark was selected for its high tactile density and known 'Golden Path', allowing for simultaneous verification of physical piece-inference accuracy and agent response quality

5.2 Results of Evaluation

Below discusses the tangible results owning to the methods outlined above.

5.2.1 `HA_Agent` vs `NA_Agent`

The agents speed when deciding a move was measured using two games, each capped at 40 moves. Below is a table detailing the mean time for move selection(μ) as well as the standard deviation between times(σ)

Comparative Latency Analysis of Move Selection ($N = 40$)

Metric	NA (White)	HA (Black)	NA (Black)	HA (White)
Mean Time (μ)	1.50s	35.4s	1.52s	20.7s
Std Dev (σ)	0.17s	13.8s	0.20s	11.7s

From the results above it can clearly be seen that `NA_Agent` is the clear winner of the speed benchmark, with `HA_Agent` lagging behind significantly. While `HA_Agent` is slightly faster when starting from white, `NA_Agent` still clearly trumps `HA_Agent` in speed and consistency and will be our preferred agent for the rest of testing.

5.2.2 Neural network testing

Below is a table detailing value, value-sign, and policy accuracy for epoch ranges 5-10. Testing was carried on an RTX 2060 GPU, leveraging CUDA cores computation time is sped up significantly by a factor of roughly $17\times$ (when compared against ARM Silicon M1 CPU chip). The results below are subject to high variance prior to policy gradient refinement, which is not unexpected as this phase serves as a precursor Reinforcement Learning optimisations not present in this paper.

Neural Network Training Metrics: Policy and Value Accuracy

Epoch	Value Acc.	Val-Sign Acc.	Policy (Top-1)	Policy (Top-5)
Epoch 5	72.30%	64.20%	31.23%	66.07%
Epoch 6	72.31%	64.44%	31.64%	66.80%
Epoch 7	76.32%	62.88%	31.74%	66.51%
Epoch 8	70.82%	65.30%	32.13%	67.12%
Epoch 9	72.74%	64.58%	32.23%	67.13%
Epoch 10	70.00%	65.76%	32.18%	67.25%

The 7 epoch iteration was chosen for development as it yielded the highest value accuracy (76.32%), providing a stable game evaluation foundation. It is significant that the policy (Top-5) accuracy consistently accounts for the majority of the probability distribution ($\sim$67%); this suggests the model effectively prunes illegal moves within the action space while identifying the best move from the evaluated state. Furthermore, the Policy (Top-1) performance peaks within this range, ensuring the primary move selection remains the most probable trajectory during inference.

5.2.3 Capture and response quality testing

When testing against Morphy's Opera Game the agent was able to adapt quickly and secure advantage, collapsing the aforementioned 'Golden Path'. To account for this another test was ran using the analyst mode of Chess.com (see A.3) over 20 half moves. Analyst moves were mirrored onto the physical board and agent moves were assessed using analyst forecasts.

Benchmark Analysis: Agent vs. Human

Move #	White (Human)	FEN Translation	Black (Agent)	Agent Eval	Chess.com Eval
1	e2e4	correct	e7e5	0.27	0.27
2	g1f3	correct	b8c6	0.24	0.24
3	f1b5	correct	g8f6	0.24	0.24
4	e1g1	correct	f6e4	0.29	0.29
5	f1e1	correct	e4d6	0.24	0.24
6	f3e5	correct	c6e5	0.28	0.29
7	e1e5	correct	f8e7	0.18	0.18
8	b1f3	correct	d6b5	3.08	0.00
9	f3d5	correct	d7d6	2.10	0.82
10	e5e7	correct	d8e7	4.38	2.04

The agent tracks the standard evaluation almost perfectly until move 8, where clear divergence occurs. Even though it eventually blunders, its ability to hold its own for the first several moves demonstrates a capacity for theoretical alignment with chess, even as 'creative' calculations arise.

5.3 Returning to Objectives

This subsection will review our progress relative to the objectives set out at the start of the project for each individual component and their full realisation.

5.3.1 Perception objectives

All perception objectives were achieved through the CV pipeline's ability to isolate board boundaries and features with high accuracy FEN translation. The system successfully closed the feedback loop by detecting player moves and updating internal model states, highlighting the robustness of the physical-to-digital interface.

5.3.2 Logic objectives

The two Logic objectives were met via the chess engine implementation. The use of bitboards ensured very little computational overhead when exploring board states. The robustness of the common language was facilitated through FEN string generation and the `Agent` interface.

5.3.3 Playing objectives

The targets set out for the Playing objectives were achieved during the operantionalisation of agents and their external dependencies, such as the neural network prediction model.

5.3.4 Overall pipeline integration

Ultimately, the successful integration of these components realises a real-time synchronised system characterised by low-latency state updates and high-fidelity physical-to-digital parity.

6 Discussion & Reflection

6.1 Interpreting the Results

The results confirm that this layered architecture is a viable prototype for a real-time, physical-to-digital chess interface. The accuracy of Perception based FEN string construction measured in (Results 5.2.3) demonstrate the reliability of the Perception module and overall pipeline, while the divergence and accuracies observed in (Results 5.2.2) indicates the Playing module operates with 'human-like' performance as opposed to total engine optimisation. Overall, this validates the asynchronous hardware-to-software pipeline, proving this system can maintain consistent communication without Perception latency effecting Playing decision making performance.

6.2 Reflection

If the development cycle were to be re-initialised, the architecture of the Playing module would shift from its current multi-agent approach to a fully data centric implementation. This would involve only developing for the `NA_Agent`, refining the dataset to embed better value-head evaluations as well as end-game execution possibly aided by Reinforcement Learning. Assuming these changes were applied to the Playing module, development could allot more time towards refining the rigour of the Perception module in harsh lighting or imperfect camera angles.

6.3 Challenges

The primary technical hurdle pertained to the sensitivity of square-occupancy within the Perception module. Harsh lighting produces shadows that introduce high-frequency gradients into the image state. These gradients often exceed the calibrated edge-density thresholds established for piece detection, causing the system to erroneously flag empty squares as occupied. Bridging this gap requires the implementation of a robust validation layer or greater experimentation with the hard sigmoid function used for thresholding.

6.4 Limitations

Beyond mechanical constraints of overhead camera capture, the system's primary limitation is its environmental sensitivity. The dependency on balanced, uniform lighting means the current hard sigmoid thresholding is not globally optimised for varied real-world lux levels. As shown below:

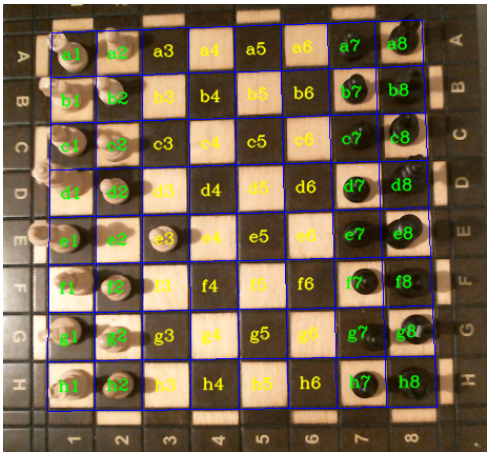


Figure 10: Example of false positive caused by harsh shadows

Furthermore, The custom dataset used to train the external chess prediction within the Playing module reveals a bottleneck in the value-head evaluation rather than the move selection model. While the AlphaZero style policy demonstrates a high degree of strategic intent, the agent's scalar value output often suffers from a drift as it is only based on if that board is winning/losing. Without a more granular training signal, like Heuristic Distillation within the dataset, the value-head struggles to differentiate between a stable and volatile advantage. This behaviour is most observable during endgame board states.

6.5 Future Work

To address this, future implementations should focus on value-head augmentation through the integration of mobility maps and centipawn evaluation of every board state. By encoding piece-activity and square-control heuristics directly into the value labels, the `NA_Agent` could learn a more nuanced "positional weight" that transcends simple win/loss outcomes.

To further enhance the robustness of the Perception module, future development should prioritise refined feature-engineering of the current system, such as synthetic data augmentation as opposed to object-detection models like YOLO. Such "silver-bullet" solutions often bypass the structural geometry inherent to the board state along with significant training and computational overhead. By hardening the current system against simulated specular noise or shadows, it can achieve the necessary environmental resistance while still maintaining mathematical transparency and efficiency required during real-time Cyber-Physical interaction.

7 Conclusion

This project has successfully developed a low-latency, physical-to-digital interface capable of bridging complex environmental information with AI agent architecture. While ambient challenges like specular noise or evaluation limitations of `NA_Agent`'s value-head were identified, the prototype validates the core objective: a robust, real-time, perceiving and playing loop. This research provides a scalable foundation for future physical AI interaction, where the precision of classical computer vision techniques meets the potential of neural network driven strategy.

References

[1] Git Statistics: Market Data Report 2026.

[2] Opera Game | Paul Morphy vs. Duke Karl (1858).

[3] L. Abeykoon, V. Patel, G. Senthilvelan, and D. Kasundra. CVChess: A Deep Learning Framework for Converting Chessboard Images to Forsyth-Edwards Notation, Nov. 2025. arXiv:2511.11522 [cs].

[4] H. Agrawal and K. Desai. CANNY EDGE DETECTION: A COMPREHENSIVE REVIEW. *International Journal of Technical Research & Science*, 9(Spl):27–35, June 2024.

[5] M. Conner and R. Talvi. YOLO for Chess Detection An Implementation of the YOLO Algorithm.

[6] N. Fierz. python-chess: A Chess Library for Python. `https://python-chess.readthedocs.io/`, 2025. Accessed: 2026-03-21.

[7] A. Liu, Y. Liang, J. Liu, G. V. d. Broeck, and J. Chen. On Effective Parallelization of Monte Carlo Tree Search, Oct. 2020. arXiv:2006.08785 [cs].

[8] D. R. Rasmussen. Parallel Chess Searching and Bitboards.

[9] J. Redmon, S. Divvala, R. Girshick, and A. Farhadi. You Only Look Once: Unified, Real-Time Object Detection, May 2016. arXiv:1506.02640 [cs].

[10] W. Rong, Z. Li, W. Zhang, and L. Sun. An Improved Canny Edge Detection Algorithm.

[11] C. E. Shannon. XXII. Programming a computer for playing chess. *The London, Edinburgh, and Dublin Philosophical Magazine and Journal of Science*, 41(314):256–275, Mar. 1950.

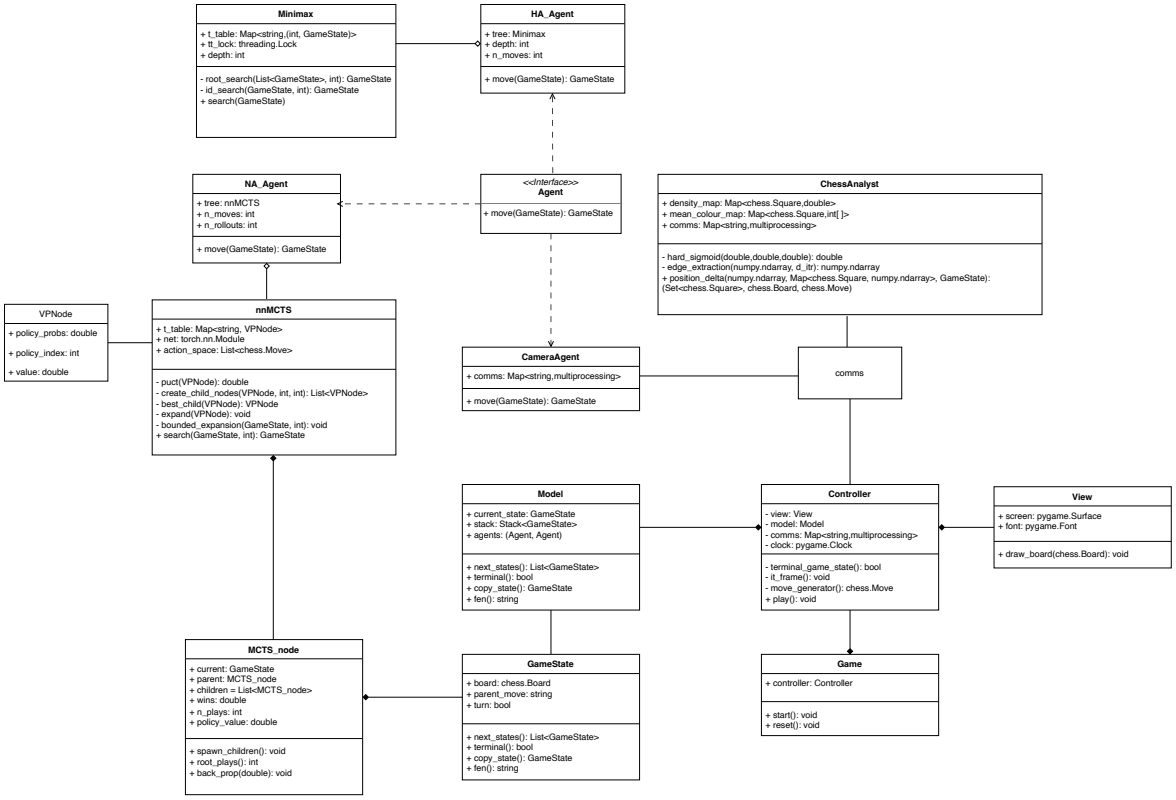
[12] A. Säfström. A Comparative Study of Parallel Search Techniques in a Chess Engine: Young Brothers Wait Concept or Lazy SMP? *KTH School of Electrical Engineering and Computer Science*.

[13] M. Tice. Checkers and chess. In *Insurgence*, pages 12–17. Routledge, 1 edition, Jan. 2020.

[14] M. Świechowski, K. Godlewski, B. Sawicki, and J. Mańdziuk. Monte Carlo Tree Search: a review of recent modifications and applications. *Artificial Intelligence Review*, 56(3):2497–2562, Mar. 2023.

A Appendix

A.1 Full UML Diagram



A.2 LiChess Kaggle dataset

Dataset of 20,000 games: kaggle.com/datasets/datasnaek/chess

A.3 Chess.com PGN analyst

Link to board analyser: chess.com/analysis